Vulnerability Assessment Report

Imagur Vulnerability Assessment Report

Imagur
132 Morphett Rd
Glengowrie SA 5044, Australia

Flinders University
Sturt Road, Bedford Park SA 5042

October 5, 2026

Report Version: 1.0

CONFIDENTIAL

Flinders University

Table of Contents

List of Figures 4

1 Document Control 5

1.1 Team 5

1.2 List of Changes 5

2 Executive Summary 6

2.1 Overview 6

2.2 Identified Vulnerabilities 7

3 Methodology 8

3.1 Objective 8

3.2 Scope 8

3.3 User Accounts and Permissions 9

4 Findings 10

C1: AWS access keys shipped in public JavaScript 10

H1: Public Amazon S3 bucket exposes all stored images, including private ones 11

M1: Hidden user_id lets one account upload into another account 12

M2: State-changing forms have no CSRF token 13

M3: Same login application exposed on cleartext HTTP port 8080 14

M4: Recursive DNS resolver open on the public server 15

M5: Image filenames are not safely encoded in every page location 16

M6: Session cookie missing Secure, HttpOnly, and SameSite 17

M7: Missing HSTS and Content-Security-Policy 18

I1: PHP version disclosed in HTTP headers 19

I2: PostgreSQL reachable on the public internet 20

I3: SSH service exposed on the public VPS 21

5 Disclaimer 22

A Appendix 23

A.1 Static Appendix Section 23
A.2 Methodology 23
A.3 Limitations 23
A.4 Coverage 23

List of Figures

Figure 1 - Distribution of identified vulnerabilities 7

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Final report	Oct 5, 2026

2 Executive Summary

2.1 Overview

Executive Summary

Imagur operates a public image-sharing website at **imagesharer.xyz** (shown to visitors as **Imager**). This report covers an authorized security assessment of that website and its approved server, performed on **5 October 2026** with the owner's permission.

Overall result

The assessment confirmed **12 findings**. In the source write-up they were rated **1 critical**, **2 high**, **6 medium**, **1 low**, and **2 informational**. This report keeps those original CVSS vectors. The findings list therefore scores some items differently (cloud keys in public JavaScript score as critical; the open S3 bucket scores as high; PostgreSQL with no scored confidentiality, integrity, or availability impact scores as informational).

The most urgent business issue remains that uploaded files live in an Amazon S3 bucket that anyone on the internet can list and download from. Images the website treats as private are still readable directly from storage.

What this means for the business

- **Private images are not private in storage.** The site's private setting does not stop a direct download from the storage bucket.
- **One signed-in user can plant files in another user's library** by changing a hidden form field on the upload page.
- **Cloud access keys are in a public JavaScript file** that anyone can download. Those keys were not used against Amazon APIs during this assessment, so whether they still work was not proven. They should still be removed from the site and rotated.
- **The server is open more than it needs to be.** Recursive DNS, PostgreSQL, and a cleartext copy of the login application on port 8080 are reachable from the internet.

Highest-priority actions

1. Lock the S3 bucket so the public cannot list or download objects. Serve files only through short-lived signed links or the application.
2. Rotate and disable the exposed cloud keys, and remove them from frontend code.
3. Bind uploads to the logged-in user. Ignore any client-supplied user ID.
4. Close PostgreSQL, recursive DNS, and HTTP port 8080 from the internet.

The remaining medium, low, and informational items (session-cookie flags, missing CSRF tokens, missing HSTS/CSP, HTML encoding of filenames, and service banners) should be fixed in the next hardening cycle.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	AWS access keys shipped in public JavaScript	10
H1	7.5	Public Amazon S3 bucket exposes all stored images, including private ones	11
M1	6.5	Hidden user_id lets one account upload into another account	12
M2	6.5	State-changing forms have no CSRF token	13
M3	5.9	Same login application exposed on cleartext HTTP port 8080	14
M4	5.8	Recursive DNS resolver open on the public server	15
M5	5.4	Image filenames are not safely encoded in every page location	16
M6	5.4	Session cookie missing Secure, HttpOnly, and SameSite	17
M7	4.2	Missing HSTS and Content-Security-Policy	18
I1	0.0	PHP version disclosed in HTTP headers	19
I2	0.0	PostgreSQL reachable on the public internet	20
I3	0.0	SSH service exposed on the public VPS	21

Vulnerability Overview

In the course of this penetration test **1 Critical**, **1 High**, **7 Medium** and **3 Info** vulnerabilities were identified:

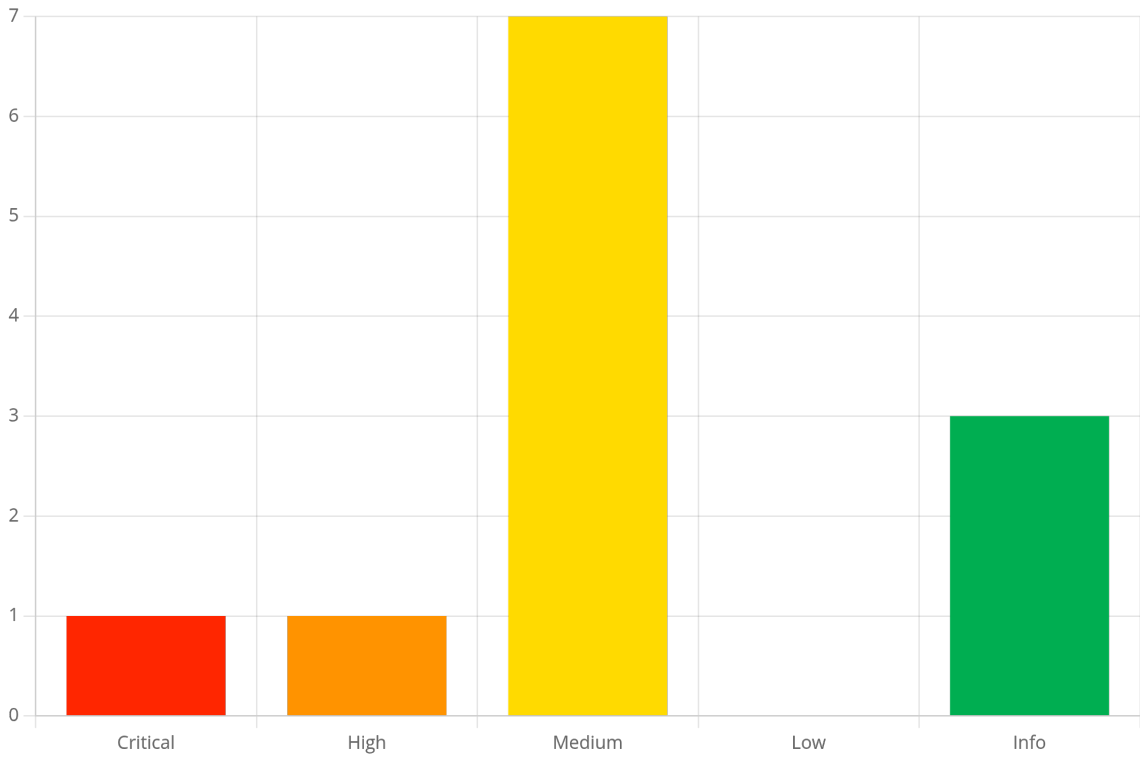


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

This assessment covered only owner-approved, in-scope assets for Imagur. Testing was performed on 5 October 2026 after authorization was verified on the main public website.

In-scope (owner approved)

System	Description
imagesharer.xyz	Root domain of the image-sharing site
https://imagesharer.xyz/	Public HTTPS application (branded Imager)
51.161.131.44	OVH VPS hosting the site (this IP only)

Observed during testing, not a separately approved scan target

Amazon S3 bucket `filesharer-research` (region ap-southeast-2) is used by the in-scope application to store uploaded images. It is mentioned in findings because the approved website writes objects there. Other customers' objects were not downloaded.

Enumerated but owner-declined / out of scope

- www.imagesharer.xyz
- resources.imagesharer.xyz
- imagur.xyz
- fshare.xyz
- OVH ASN/netblocks except 51.161.131.44
- CloudFront, Spaceship DNS, Flinders University, Google CDN

Allowed methods

Authorization check, passive and low-impact enumeration, owner validation of assets, port discovery on 51.161.131.44, version scan of open ports, web mapping, authenticated testing with tester-created accounts, and safe injection proofs only.

Forbidden methods (not used)

Brute force, denial of service, malware, phishing, credential stuffing, password spraying, MFA bypass, lateral movement, destructive changes, and downloading other customers' S3 objects.

3.3 User Accounts and Permissions

Authenticated testing used tester-created accounts registered through the public website. No owner-provided staff credentials were used. Account passwords, session cookie values, and cloud secret keys are not included in this report.

4 Findings

C1: AWS access keys shipped in public JavaScript	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	https://imagesharer.xyz/js/awsvalid.js
References	• CWE-798 • CWE-200 • OWASP A07 Identification and Authentication Failures

Overview

Cloud access-key material is hardcoded in a JavaScript file that anyone can download from the website.

Details

https://imagesharer.xyz/js/awsvalid.js is reachable without login. It embeds AWS accessKeyId (AKIA-format, 20 characters) and secretAccessKey as string literals and calls STS at https://sts.amazonaws.com/ from the browser (validateAwsCredentials, region us-east-1). The same access key id also appears in S3 presigned URLs on image pages. Live STS/IAM use of the secret was not performed, so key validity was not independently proven. The presence of long-lived IAM material in a public static file is still a credential exposure. Secret values and full access keys are not included in this report.

Recommendation

Remove the file from the site. Create credentials only on a server, never in frontend code. Rotate and disable the exposed IAM user immediately. Use IAM roles or short-lived tokens. Check CloudTrail for use of this key.

H1: Public Amazon S3 bucket exposes all stored images, including private ones	
Score	7.5 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Target	• https://imagesharer.xyz/ • S3 bucket filesharer-research (ap-southeast-2)
References	• CWE-284 • CWE-552 • OWASP A01 Broken Access Control

Overview

Anyone on the internet can list and download files from the image storage bucket. Images marked private in the website are still readable directly from storage.

Details

Uploading a test image caused the application to serve objects from Amazon S3 bucket filesharer-research (ap-southeast-2) under a predictable users//.jpg key. An unauthenticated GET of our own object returned HTTP 200 with Content-Type image/jpeg. An unauthenticated GET of the bucket root returned an object listing (about 1000 keys). A second test image uploaded with is_public=false was protected on the website (unauthenticated requests to /images/ redirected away), but the same object remained anonymously readable on S3. The private flag therefore does not protect the file. Other customers' objects were not downloaded.

Recommendation

Block public list and public read on the bucket. Serve objects only through short-lived, least-privilege signed URLs or a backend proxy. Enforce private objects at the storage ACL/IAM layer, not only in the PHP page. Rotate any credentials that could have been used with this bucket and review access logs.

M1: Hidden user_id lets one account upload into another account	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
Target	https://imagesharer.xyz/images/upload
References	• CWE-639 • CWE-284 • OWASP A01 Broken Access Control

Overview

A logged-in user can attach files to someone else's library by changing a hidden form field.

Details

POST /images/upload accepts multipart field user_id from the client (set to the caller's UUID in HTML, but not bound to the session server-side). A second test account uploaded a JPEG while sending the first account's UUID. The server accepted it and created a new image. The object appeared on the first account's dashboard, the image page showed the first account as owner, and the file was stored under the first account's S3 prefix. Unauthenticated upload was rejected (302 to /login).

Recommendation

Ignore client-supplied user_id. Bind ownership to the authenticated session. Authorize every upload, visibility change, and delete against that session user.

M2: State-changing forms have no CSRF token	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N
Target	• https://imagesharer.xyz/images/upload • https://imagesharer.xyz/images/{id}/delete • https://imagesharer.xyz/images/{id}/visibility • https://imagesharer.xyz/profile/avatar • https://imagesharer.xyz/logout • https://imagesharer.xyz/register
References	• CWE-352 • OWASP A01 Broken Access Control

Overview

Logged-in actions such as upload, delete, visibility change, avatar change, and logout do not include an anti-CSRF token.

Details

Upload, image delete, visibility, avatar, logout, and registration forms had no hidden CSRF field and no CSRF token in HTML. Combined with a session cookie that may be sent on cross-site requests (no SameSite on PHPSESSID), a third-party page could submit those POSTs in a victim browser.

Recommendation

Issue a per-session CSRF token, require it on every cookie-authenticated POST, and set SameSite on the session cookie. Prefer re-auth for delete.

M3: Same login application exposed on cleartext HTTP port 8080	
Score	5.9 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:L/A:N
Target	51.161.131.44:8080
References	• CWE-319 • OWASP A02 Cryptographic Failures

Overview

The Imager login page is also served unencrypted on port 8080.

Details

Port 8080 on the approved server runs Apache/2.4.68 (Debian) with X-Powered-By PHP/8.2.34. GET http://51.161.131.44:8080/login returned 200 and the same "Log in — Imager" form (email/password POST to /login). Credentials and session cookies sent to this port travel in cleartext. PHPSESSID lacked the Secure flag, so browsers may send it here.

Recommendation

Disable the Apache listener on 8080 or restrict it to localhost. Redirect all clients to HTTPS on 443. HSTS after HTTPS is clean.

M4: Recursive DNS resolver open on the public server	
Score	5.8 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:N/I:N/A:L
Target	51.161.131.44:53
References	• CWE-406 • US-CERT TA13-088A

Overview

The website server will answer DNS lookups for unrelated names from the internet.

Details

TCP/53 is open on 51.161.131.44. A recursive query for example.com A against this resolver returned NOERROR with A records, flags rd and ra set. That is an open recursive resolver. It can be abused for amplification and cache issues. Only a single lookup was used as proof.

Recommendation

Do not expose a recursive resolver on a public VPS. Bind DNS to localhost if local resolution is needed, or disable the service. Restrict to internal networks if it must remain.

M5: Image filenames are not safely encoded in every page location	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N
Target	https://imagesharer.xyz/images/{id}
References	• CWE-79 • OWASP A03 Injection

Overview

A crafted file name can inject HTML into the image page.

Details

A JPEG uploaded with filename clawsecxss.jpg was accepted. On the resulting image page, the title, heading, and alt text encoded the brackets as <img>, but at least one span output the raw characters . That is stored HTML injection. A more complex breakout filename was rejected at upload, so script execution was not fully proven. Encoding is inconsistent and unsafe.

Recommendation

HTML-encode all filename/title output in every template context. Prefer storing a generated display name. Add a Content-Security-Policy that disallows inline script.

M6: Session cookie missing Secure, HttpOnly, and SameSite	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	https://imagesharer.xyz/ (PHPSESSID)
References	• CWE-614 • CWE-1004 • OWASP A05 Security Misconfiguration

Overview

The login session cookie can leak to scripts and to unencrypted HTTP.

Details

A fresh GET https://imagesharer.xyz/login set PHPSESSID with Path=/ only. Secure, HttpOnly, and SameSite were absent. A second cookie, page_views, correctly used Secure, HttpOnly, and SameSite=Lax. The session cookie is the weaker of the two. The same application is also offered on HTTP port 8080, so a missing Secure flag is more serious. Cookie values are not included in this report.

Recommendation

Set PHPSESSID with Secure; HttpOnly; SameSite=Lax (or Strict). Enable session.cookie_secure, session.cookie_httponly, and session.cookie_samesite in PHP. Do not serve the app on cleartext HTTP.

M7: Missing HSTS and Content-Security-Policy	
Score	4.2 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	https://imagesharer.xyz/
References	• CWE-319 • CWE-1021 • OWASP A05 Security Misconfiguration

Overview

The HTTPS site does not tell browsers to stay on HTTPS or limit script sources.

Details

GET https://imagesharer.xyz/login returned X-Content-Type-Options nosniff, X-Frame-Options DENY, X-XSS-Protection, and Referrer-Policy, but no Strict-Transport-Security and no Content-Security-Policy. HTTP on port 80 does redirect to https://imagesharer.xyz/. Without HSTS, SSL stripping remains possible. CSP would also reduce impact of the filename HTML injection.

Recommendation

Send Strict-Transport-Security with a long max-age (and includeSubDomains once www is intended). Add a strict CSP default-src/script-src.

I1: PHP version disclosed in HTTP headers	
Score	0.0 (Info)
Vector string	N/A
Target	• https://imagesharer.xyz/ • 51.161.131.44:8080
References	CWE-200

Overview

Responses include X-Powered-By: PHP/8.2.34, which tells attackers the exact language version.

Details

Nginx on 443 and Apache on 8080 both return X-Powered-By PHP/8.2.34. Apache also returns Server: Apache/2.4.68 (Debian). This is reconnaissance, not a direct exploit.

Recommendation

Turn off expose_php, hide Server tokens, and keep PHP/Apache patched.

I2: PostgreSQL reachable on the public internet	
Score	0.0 (Info)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N
Target	51.161.131.44:5432
References	• CWE-668 • CWE-200

Overview

The database port is open to the world. A password is required, but attackers can still try to log in.

Details

Port 5432 on 51.161.131.44 is open and identified as PostgreSQL. A single unauthenticated attempt as user postgres with no password failed with password-required (not trust/peer). No password guessing was performed. Public exposure still enlarges the attack surface for the application database.

Recommendation

Bind PostgreSQL to localhost or a private network. Firewall 5432 from the internet. Require strong auth, TLS, and restrict pg_hba.conf.

I3: SSH service exposed on the public VPS	
Score	0.0 (Info)
Vector string	N/A
Target	51.161.131.44:22
References	CWE-200

Overview

SSH is open on the same public IP as the website.

Details

Port 22 is open running OpenSSH 10.0p2 Debian 7+deb13u4. No login guessing was performed. Internet-facing SSH is common on a VPS but should be locked down.

Recommendation

Restrict SSH by firewall to admin IPs, use keys only, and keep OpenSSH patched.

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section

A.2 Methodology

The assessment followed a phased, non-destructive methodology:

1. Authorization GET to the main public website before any testing.
2. Passive and low-impact enumeration of company-related assets.
3. Owner validation of in-scope versus excluded assets.
4. Port discovery on 51.161.131.44, then version detection on open ports only.
5. Web mapping of login, register, dashboard, profile, image upload, image pages, and common path checks.
6. Authenticated testing with tester-created accounts.
7. Safe injection proofs only. No brute force, denial of service, or data dump of other customers' S3 objects.

Enumeration run timestamps were recorded around 2026-10-04T23:30Z to 2026-10-04T23:37Z. Authorization and assessment are dated 2026-10-05.

A.3 Limitations

The following limitations apply to this assessment:

- www.imagesharer.xyz and resources.imagesharer.xyz were not tested (owner declined).
- No password guessing was performed.
- The AWS IAM key found in public JavaScript was not used against AWS APIs, so key validity is unproven.
- Other S3 objects were not downloaded (only a listing count and our own test objects).
- The PostgreSQL password was not attacked.
- No phone number was available for the business contact record.
- Testing was performed from a single vantage point.

A.4 Coverage

Host 51.161.131.44 ports reviewed:

- 22/tcp — SSH
- 53/tcp — DNS
- 80/tcp — HTTP
- 443/tcp — HTTPS
- 5432/tcp — PostgreSQL
- 8080/tcp — HTTP (Apache/PHP login application)

Web application coverage on https://imagesharer.xyz/:

- login, register, dashboard, profile, image upload, image pages, and common path checks

Confirmed findings in this report: 12 (1 critical, 2 high, 6 medium, 1 low, 2 informational).